

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ

УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

Кафедра «Програмна інженерія»

ЗВІТ

з лабораторної роботи №2

«Моделювання системи (UML)»

з дисципліни «Основи програмної інженерії»

Виконав:

ст. гр. ПЗПІ-25-3

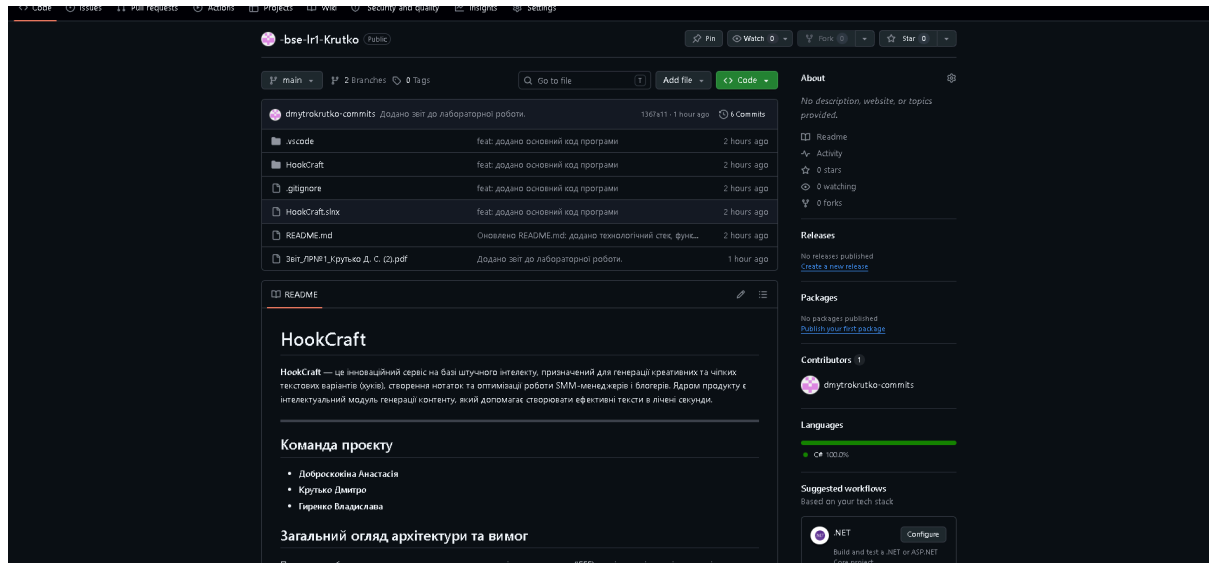
Крутько Д. С.

Прийняла:

Гребенюк В.О.

Харків 2026

Обраний проект: Hookcraft



Обрати функціональні вимоги:

ID Функціональна вимога FR-01 - Система повинна дозволяти користувачу вводити тему або опис контенту.

FR-02 - Система повинна генерувати кілька варіантів текстових «гачків» за допомогою AI.

FR-03 - Користувач повинен мати можливість обрати стиль тексту (інтригуючий, гумористичний).

FR-04 - Система повинна відображати список згенерованих варіантів тексту.

FR-05 - Користувач може скопіювати обраний текст у буфер обміну.

FR-06 - Зареєстрований користувач може переглядати історію своїх генерацій.

FR-07 - Система повинна підтримувати реєстрацію користувача через email або Google.

Побудувати діаграму прецедентів: Актори: • Гість (Guest) • Зареєстрований користувач (Registered User) • Адміністратор (Admin) Прецеденти: 1. Ввести тему (Enter Topic) 2. Обрати стиль (Choose Style) 3. Згенерувати гачки (Generate Hooks) 4. Переглянути результати (View Results) 5. Скопіювати текст (Copy Text) 6. Переглянути історію (View History) 7. Зареєструватись / увійти (Register / Login) 8. Керувати лімітами (Manage Limits) - для адміністратора

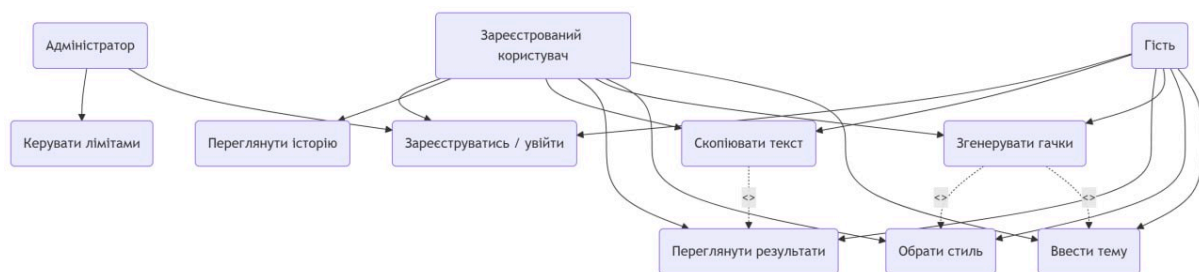


Рис.1 - Діаграма прецедентів.

Побудувати діаграму класів:

Класи:

1. User (абстрактний або базовий) – атрибути: id, email, role, registrationDate; методи: authenticate(), getHistory()

2. Guest (наслідує User) – додаткові атрибути: remainingGenerations; методи:

useTrial()

3. RegisteredUser (наслідує User) – атрибути: favoriteHooks, historyList;
методи: saveToFavorites(), viewHistory()

4. Admin (наслідує User) – методи: manageUsers(), updateLimits()

5. GenerationRequest – атрибути: topic, style, timestamp, userId; методи:
submit(), getResult()

6. Hook – атрибути: id, text, style, isFavorite, createdAt; методи:
copyToClipboard(), getText()

7. History (композиція з RegisteredUser) – атрибути: listOfHooks; методи:
addHook(), clearHistory()

8. AuthService – методи: login(email, password), register(email, password),
logout()

Зв'язки:

- Наслідування: Guest, RegisteredUser, Admin → User.
- Асоціація: User 1 – * GenerationRequest.
- Композиція: RegisteredUser 1 – * History (без користувача історія не існує).
- Агрегація: History * – * Hook (гачки можуть існувати окремо).
- Залежність: GenerationRequest ..> AuthService.

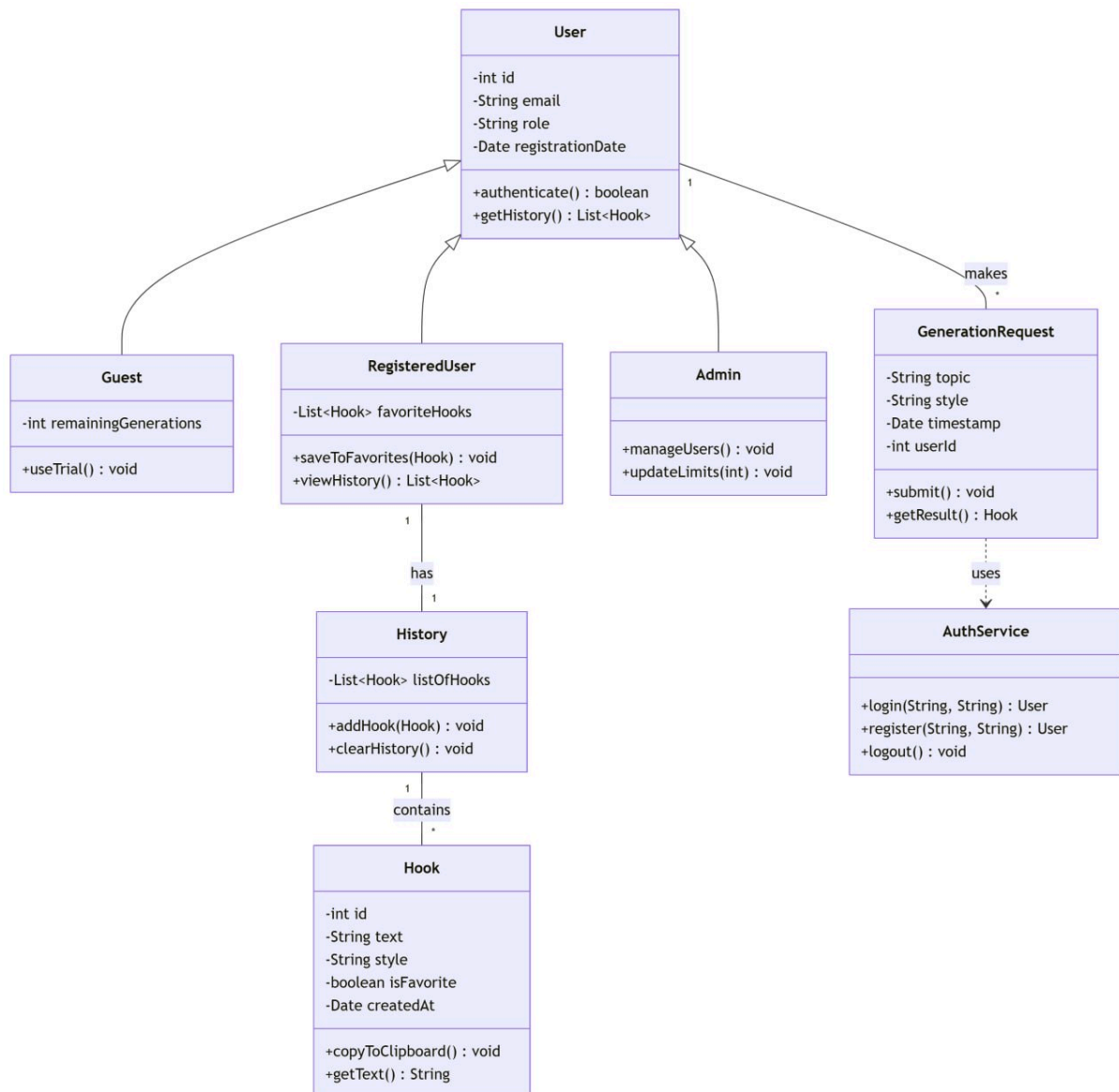


Рис 2. - Діаграма класів.

Побудувати діаграму послідовності:

Обраний сценарій: "Генерація гачків зареєстрованим користувачем" (включає FR-01, FR-02, FR-03, FR-04).

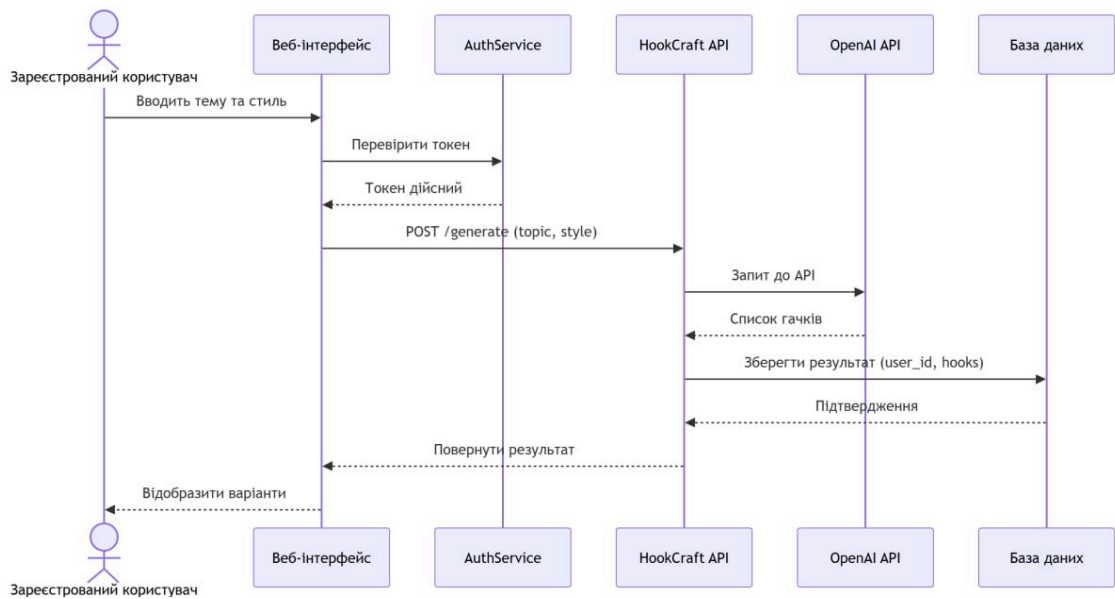


Рис 3. - Діаграма послідовності.

Скласти матрицю трасовності:

Вимога	Прецедент	Класи	Діаграма послідовності
FR-01	UC-01 "Ввести тему"	User, GenerationRequest	SD-01 (фрагмент)
FR-02	UC-03 "Згенерувати гачки"	GenerationRequest, Hook, AuthService	SD-01
FR-03	UC-02 "Обрати стиль"	GenerationRequest	SD-01 (фрагмент)
FR-04	UC-04 "Переглянути результати"	Hook, UI	SD-01 (фрагмент)
FR-05	UC-05 "Скопіювати текст"	Hook	—
FR-06	UC-06 "Переглянути історію"	RegisteredUser, History, Hook	—
FR-07	UC-07 "Зареєструватись / увійти"	AuthService, User	—

Діаграму прецедентів (Крок 3) виконав Крутько Дмитро, діаграму класів (Крок 4) виконала Доброскокіна Анастасія, діаграму послідовності (Крок 5) виконала Гиренко Владислава. Матрицю трасовності складено командою спільно.